
MLang

Direction Générale des Finances Publiques

janv. 09, 2026

Table des matières

1	Le langage M	1
1.1	Introduction au langage M	1
1.2	La syntaxe du M	4
1.3	Les fonctions	12
1.4	Les fichiers IRJ	14
1.5	Le compilateur MLang	15
2	Exemples	23
2.1	Calcul des valeurs	23
2.2	Les fonctions	26
3	Documentation développeur	35

1.1 Introduction au langage M

1.1.1 Les éléments de base

Un programme M se caractérise par la définition successive :

- de types de variables avec leurs attributs ;
- de domaines et d'événements ;
- d'espaces de variables ;
- d'applications ;
- de variables et de constantes ;
- des fonctions ;
- de règles de calcul associées ou non à une application.

Exécuter d'un tel programme pour une application donnée correspond à exécuter l'ensemble des règles de calcul associée la dite application.

L'ordre d'exécution des règles de calcul dépend des dépendances entre les variables. Si une variable est affectée dans une règle, alors cette règle sera exécutée avant toutes celles utilisant la valeur de la dite variable.

Si une variable est lue et écrite cycliquement, l'ordre d'exécution des règles n'est pas garanti.

1.1.2 Définitions par défaut

Les types de variables, leurs attributs, les domaines et les événements sont historiquement des mots clés propres au langage M.

Il est aujourd'hui possible de les définir à la main.

1.1.3 Aperçu de la syntaxe

Voici un exemple simple de programme M minimal.

```
# Fichier: test.m
```

(suite sur la page suivante)

(suite de la page précédente)

```

# On définit un espace de variable global dans lequel nos variables
# seront stockées par défaut.
espace_variangles GLOBAL : par_defaut;

# On définit deux domaines obligatoires:
# * un domaine pour les règles;
# * un domaine pour les vérificateurs.
domaine regle mon_domaine_de_regle: par_defaut: calculable;
domaine verif mon_domaine_de_verifications: par_defaut;

# On définit une ou plusieurs application pour nos règles.
application mon_application;

# On définit une cible, un ensemble d'instructions à exécuter.
cible hello_world:
application : mon_application;
afficher "Bonjour, monde!\n";

```

Cet exemple jouet n'utilise ni ne définit aucune variable, les domaines et les espaces de variables ne sont pas utilisés ici mais ils seront nécessaires plus tard.

Pour essayer notre exemple, nous allons créer un fichier “test.irj” avec le contenu suivant :

```

#NOM
MON-TEST
#ENTREES-PRIMITIF
#CONTROLES-PRIMITIF
#RESULTATS-PRIMITIF
#ENTREES-CORRECTIF
#CONTROLES-CORRECTIF
#RESULTATS-CORRECTIF
##

```

Le détail de la syntaxe irj peut être sur la page dédiée : [Les fichiers IRJ](#)

Enfin, on peut lancer mlang.

```

$ mlang --without_dfgip_m test.m -A mon_application --mpp_function hello_world --run_
↪test test.irj
Parsing: completed!
Bonjour, monde!
[RESULT] test.irj
[RESULT] No failure!
[RESULT] Test passed!

```

1.1.4 Définition de variables

Les variables sont divisées en deux catégories.

- Les variables saisies sont les variables d'entrées du programme M.
- Les variables calculées sont les variables sur lesquelles seront calculées les données.

Chacune de ces catégories principales doivent être dotées d'attributs, un ensemble d'entiers constants pour une variable donnée. Ainsi, on peut rajouter à notre programme test les lignes suivantes :

```
variable saisie : attribut mon_attribut;
variable calculee : attribut mon_attribut;
X : saisie mon_attribut = 0 alias ALIAS_DE_X : "Cette variable s'appelle X";
Y : calculee mon_attribut = 1 : "Cette variable s'appelle Y";
```

Notez que la variable X a un alias ALIAS_DE_X. Toutes les variables saisies doivent être parées d'un alias qui peut être utilisé de la même façon que son nom original. Cet alias existe initialement pour faire le lien entre la variable utilisée dans le M (nom original) et le code de la variable dans le formulaire de déclaration de l'impôt sur le revenu tel qu'on le retrouve aujourd'hui sur le site de déclaration (1AP, 8TV, ...).

Ajoutons une nouvelle cible à notre calcul :

```
cible hello_world2:
application : mon_application;
afficher "Bonjour, monde, X = ";
afficher (X);
afficher " !\n";
Y = X + 1;
afficher "Y = ";
afficher (Y);
afficher " !\n";
```

Et lançons le calcul :

```
$ mlang --without_dfgip_m test.m -A mon_application --mpp_function hello_world2 --run_
↪test test.irj
Parsing: completed!
Bonjour, monde, X = indefini !
Y = 1 !
[RESULT] test.irj
[RESULT] No failure!
[RESULT] Test passed!
```

Pour comprendre la valeur finale de Y, référez-vous à la section [Les valeurs](#).

1.1.5 Règles de calcul

Les règles en M sont des unités de calcul de variables associées à une ou plusieurs application et optionnellement à un domaine de règles. Elles sont composées d'une succession d'affectations. Voici la définition de deux règles simples que nous rajoutons à notre fichier test :

```
Z : calculee mon_attribut = 1 : "Cette variable s'appelle Z";

regle mon_domaine_de_regles 1:
application : mon_application;
Z = Y + 1;

regle 2:
application : mon_application;
Y = X + 1;
```

La première règle est associée au domaine mon_domaine_de_regles tandis que la seconde n'étant pas spécifié, sera associée au domaine de règle par défaut. Dans notre exemple, il s'agissait également de mon_domaine_de_regles.

Le calcul d'un domaine correspond au calcul de l'ensemble de ses règles. L'ordre d'application des règles dépend de l'ordre d'affectation des variables. Dans notre cas, **X** est une entrée dont **Y** dépend (règle 2), et **Z** dépend de **Y**.

Par conséquent, la règle 2 sera appliquée avant la règle 1. On peut ainsi rajouter une cible qui calcule le domaine de règles :

```
cible calc_test:
application : mon_application;
calculer domaine mon_domaine_de_regles;
afficher "X = ";
afficher (X);
afficher "\nY = ";
afficher (Y);
afficher "\nZ = ";
afficher (Z);
afficher "\n";
```

Et lancer le calcul :

```
$ mlang --without_dfgip_m test.m -A mon_application --mpp_function calc_test --run_test_
↪test.irj
Parsing: completed!
X = indefini
Y = 1
Z = 2
[RESULT] test.irj
[RESULT] No failure!
[RESULT] Test passed!
```

1.2 La syntaxe du M

1.2.1 Programme M et applications

Un programme M est formé d'une suite de caractères codés sur 8 bits. Les 128 premiers codes de caractères correspondent aux codes ASCII. Un programme M est constitué d'une suite d'éléments parmi lesquels on compte :

- des déclarations d'applications;
- des définitions de constantes;
- des déclarations d'enchaineurs;
- des définitions de catégories de variables;
- des déclarations de variables;
- des déclarations d'erreurs;
- des déclarations de fonctions externes;
- des définitions de domaines de règles;
- des définitions de domaines de vérifications;
- des déclarations de sorties;
- des règles;
- des vérifications;
- des fonctions;
- des cibles.

Un programme M peut définir plusieurs applications. Pour être traité (compilation, interprétation, etc.), il nécessite la donnée d'une liste d'applications. Cette liste est typiquement fournie comme argument du compilateur, de l'interpréteur ou de tout autre outil de traitement. Cette liste sera appelée la liste des applications sélectionnées.

1.2.2 Définitions liminaires

Les formes de Backus-Naur étendues sont utilisées pour préciser la morphologie du langage. Un lexème est une suite de caractères et chaque forme est susceptible d'en reconnaître un ensemble.

Ces formes sont étendues avec les notations suivantes :

- `<non-terminal:identifiant>` représente le non-terminal, pour lequel la chaîne de caractères qu'il reconnaît est représentée par `identifiant`;
- `forme 1 ^ ... ^ forme n` qui représente une suite de lexèmes correspondants aux formes 1 à n, dans n'importe quel ordre ;
- `et : (forme 0) séparateur ...` qui représente une liste non-vide de lexèmes reconnus par la forme 0, tous séparés par des lexèmes correspondants au séparateur.

Les lexèmes suivants sont les mots réservés : **BOOLEEN, DATE_AAAA, DATE_JJMMAAAA, DATE_MM, ENTIER, REEL, afficher, afficher_erreur, aiguillage, ajouter, alias, alors, anomalie, application, apres, argument, arranger_evenements, attribut, autorise, avec, base, calculable, calculee, calculer, cas, categorie, champ_evenement, cible, const, dans, dans_domaine, discordance, domaine, enchaineur, entre, erreur, espace, espace_variables, et, evenement, evenements, exporte_erreurs, faire, filtrer, finalise_erreurs, finquand, finsi, fonction, increment, indefini, indenter, informative, iterer, leve_erreur, meme_variable, nb_anomalies, nb_bloquantes, nb_categorie, nb_discordances, nb_informatives, neant, nettoie_erreurs, nettoie_erreurs_finalisees, nom, non, numero_compl, numero_verif, ou, par_defaut, pour, puis_quand, quand, reference, regle, restaurer, restituee, resultat, saisie, si, sinon, sinon_si, sortie, specialise, stop, tableau, taille, trier, type, un, valeur variable, verif, verifiable, et verifier.**

Les lexèmes numériques sont définis comme suit :

- `<naturel> ::= [0-9] [0-9 _]*`
- `<réel> ::= <naturel> ( . <naturel> )?`
- `<symbole>` est la forme `[a-z A-Z 0-9 _]+` dont sont exclus `<naturel>`s et les mots réservés ;
- `<variable>` est un `<symbole>` que l'on distingue pour représenter les mots pouvant prendre le nom d'une constante.

On définit également les atomes numériques, prenant une valeur soit par un nombre littéral, soit par un symbole représentant une constante ou une variable :

- `<atome naturel> ::= <naturel> | <variable>`
- `<atome reel> ::= <réel> | <variable>`

Les chaînes de caractères littérales ont la forme suivante :

- `<chaîne> ::= " [^ "]* "`

Déclaration une d'application

Les *déclarations d'applications* ont la forme suivante :

```
application <symbole>;
```

où `<symbole>` est le nom de l'application déclarée.

Définition d'une constante

Les *définitions de constantes* ont la forme suivante :

```
<symbole> : const = <atome réel> ;
```

Le symbole est le nom de la constante.

Déclaration d'un enchaîneur

Les *déclarations d'enchaineurs* ont la forme suivante :

```
enchaineur <symbole> : <symboles> ;
```

avec : <symboles> ::= <symbole> , ... représente l'ensemble des applications incluant cet enchaîneur. Le <symbole> de l'enchaineur est le nom de l'application déclarée.

Définition d'une catégorie de variables

Les *définitions de catégories de variables* ont la forme suivante :

```
variable (saisie | calculee) <nom> (: attribut <symboles>)? ;
```

avec :

- <nom> ::= <symbole>+ est le nom de la catégorie de variables;
- <symboles> ::= <symbole> , ... représente l'ensemble de ses attributs.

Déclaration d'une variable

La forme des *types de variables* est comme suit :

```
<type variable> ::= BOOLEEN | DATE_AAAA | DATE_JJMMAAAA | DATE_MM | ENTIER | REEL
```

Variable saisie

Les *déclarations de variables saisies* ont la forme suivante :

```
<symbole> : saisie <catégorie> <attributs> alias <symbole> : <chaîne> type <type_↵  
↵variable> ;
```

avec :

- <catégorie> ::= <symbole>+
- <attributs> ::= <attribut> restituee? <attribut>
- <attribut> ::= <symbole> = <atome naturel>

Le <symbole> est le nom de la variable.

Variable calculée

Les *déclarations de variables calculées* ont la forme suivante :

```
<symbole>  
: (table [ <atome naturel> ])? calculee (base? ^ restituee?)  
: <chaîne> type <type variable> ;
```

Le est le nom de la variable. Si c'est un tableau, l' est sa taille.

Déclaration d'une erreur

Les *déclarations d'erreurs* ont la forme suivante :

```
<symbole> : <type erreur> : <chaîne> : <chaîne> : <chaîne> : <chaîne> : <chaîne> ;
```

avec :

- <type erreur> ::= anomalie | discordance | informative

Le symbole est le nom de l'erreur.

Déclaration d'une fonction externe

Les *déclarations de fonctions* externes ont la forme suivante :

```
<symbole> : fonction <atome naturel> ;
```

Le <symbole> est le nom de la fonction, et l' son arité.

Définition d'un domaine de règles

Les *définitions de domaines de règles* ont la forme suivante :

```
domaine regle <nom> <paramètres> ;
```

avec :

- <nom> ::= <symbole>+ est le nom du domaine de règles.
- <paramètres> ::= (: specialise)? ^ (: calculable)? ^ (: par_default)?
- ::= , ...` représente l'ensemble des domaines de règles que spécialise ce domaine.

Définition d'un domaine de vérifications

Les *définitions de domaines de vérifications* ont la forme suivante :

```
domaine verif <nom> <paramètres> ;
```

avec :

- <nom> ::= <symbole>+ est le nom domaine de vérifications que spécialise ce domaine.
- <paramètres> ::= (: specialise <noms>)? ^ (: autorise <catvars>)? ^ (: par_default)? ^ (: verifiable)?
- <noms> ::= <nom> , ...
- <catvars> ::= * | saisie (* | <nom>) | calculée (* | base)? représente les catégories des variables autorisées à être utilisées dans les vérifications de ce domaine.

Déclaration d'une sortie

Les *déclarations de sorties* ont la forme suivante :

```
sortie ( <symbole> ) ;
```

Indices

Les *indices* ont la forme suivante :

```
<indices> ::= <indice> ; ...
<indice> ::= <minuscule> = <intervalle> , ...
```

avec :

- <minuscule> ::= [a-z]
- <majuscule> ::= [A-Z]
- <intervalle> ::= <majuscule>+ | <majuscule> .. <majuscule> | <naturel> (.. <naturel> | - <variable>)?

Expressions

Les *expressions atomiques* ont la forme suivante :

```
<atomique booléen> ::=
( <expression booléenne> )
| <expression numérique> (<= | >= | < | > | != | =) <expression numérique>
| <expression numérique> non? dans ( <intervalles numériques> )
| (present | non_present) ( <symbole> ([ <expression numérique> ])? )

<atomique numérique> ::=
| <expression numérique>
| <atome réel>
| <symbole> [ <expression numérique> ]
| <symbole> ( (<expression numérique> , ...)? )
| somme ( <indices> : <expression numérique> )
| si ( <expression booléenne> )
  alors ( <expression numérique> )
  (( sinon <expression numérique> ))?
fin si
```

avec :

```
— <expression ou> ::= <expression et> | <expression ou> ou <expression ou>
— <expression et> ::= <expression non> | <expression et> et <expression et>
— <expression non> ::= <atomique booléen> | non <expression non>
— <expression + -> ::= <expression * /> | <expression + -> (+ | -) <expression + ->
— <expression * /> ::= <expression -> | <expression * /> (* | /) <expression * />
— <expression -> ::= <atomique numérique> | - <expression ->
— <intervalles numériques> ::= <intervalle numérique> , ...
— <intervalle numérique> ::= <expression numérique> (.. <expression numérique>)?
```

Les formules ont la forme suivante :

```
<formule> ::=
  <symbole> ([ <expression numérique> ])? = <expression numérique>
```

Les multi-formules ont la forme suivante :

```
<multi-formule> ::= pour <indices> : <formule>
```

Instructions

Les *instructions* ont la forme suivante :

```
<instruction> ::=
  <formule>
  | <multi-formule>
  | si <expression~booléenne>
    alors <instruction>+
    (sinon <instruction>+)?
  fin si
  | calculer domaine <symbole>+;
  | calculer enchaineur <symbole>;
  | calculer cible <symbole>
  (: avec <atome~réel> , ...)?;
```

(suite sur la page suivante)

(suite de la page précédente)

```

| verifier domaine <symbole>+
  (: avec <expression~booléenne>);
| (afficher | afficher__erreur) <affichable>+;
| iterer
  : variable <symbole>
  : categorie <symbole>+ , ...
  (: avec <expression~booléenne>)?
  : dans ( <instruction>+ )
| restaurer
  (
  : <symbole> , ...
  | : variable <symbole> : categorie <symbole>+ , ...
    (: avec <expression~booléenne>)?
  )+
  : apres ( <instruction>+ )
| leve__erreur <symbole> <symbole>?;
| (
  nettoie__erreurs
  | exporte__erreurs
  | finalise__erreurs
  ) ;
| aiguillage (nom)? (<symbole>) : (<multiple_cas>)
| stop <type_stop>?

```

avec :

```

<affichable> ::=
  <chaîne>
  | `*(` <expression~numérique> `)*`
  | (*:*` <atome~naturel> (*..*` <atome~naturel>)?)?
  | (*nom*` | `*alias*`) `*(` <symbole> `)*`
  | `*indenter*` `*(` <atome~naturel> `)*`

<multiple cas> ::= <cas>*

<cas> ::=
  cas <atome reel> : <instruction>+
  | cas indefini : <instruction>+
  | cas <symbole> : <instruction>+
  | par_default : <instruction>+

<type_stop> ::=
  application
  | cible
  | fonction
  | <symbole>

```

Les *instructions simples* ont la forme suivante :

```

<instruction simple> ::=
  <formule>
  | <multi-formule>

```

(suite sur la page suivante)

```
| `*si*` <expression~booléenne>
  `*alors*` <instruction~simple>+
  (`*sinon*` <instruction~simple>+)?
  `*finsi*`
| (`*afficher*` | `*afficher__erreur*`) <affichable>+ `*;`
```

Définition d'une règle

Les *règles* ont la forme suivante :

```
<regle> ::=
  regle <symbole>.. <naturel> :
  (
    application : <symbole> , ... ;
    ^^ (enchaineur : <symbole> , ... ;)?
    ^^ (variable temporaire : <déclaration> , ... ;)?
  )
  <instruction~simple>+
```

avec :

— <déclaration> ::= <symbole> ([<atome~naturel>])?

Définition d'une vérification

Les *vérifications* ont la forme suivante :

```
<vérification> ::=
  verif <symbole> <naturel> :
  application : <symbole> , ... ;
  si <expression~booléenne>
  alors erreur <symbole> <symbole>?
  ;
```

Définition d'une fonction

Les *fonctions* ont la forme suivante :

```
<fonction> ::=
  fonction <symbole> :
  (
    application : <symbole> , ... ;
    ^^ (variable temporaire : <déclaration> , ... ;)?
    ^^ (argument : <symbole> , ... ;)?
    ^^ resultat : <symbole> ;
  )
  <instruction~simple>+
```

avec :

— <déclaration> ::= <symbole> ([<atome~naturel>])?

Définition d'une cible

Les *cibles* ont la forme suivante :

```
<cible> ::=
  cible <symbole> :
  (
    application : <symbole> , ... ;
    ^^ (enchaineur : <symbole> , ... ;)?
    ^^ (variable temporaire : <déclaration> , ... ;)?
    ^^ (argument : <symbole> , ... ;)?
  )
  <instruction>+
```

avec :

— <déclaration> ::= <symbole> ([<atome~naturel>])?

Commentaires

Les commentaires sont précédés du caractère #. Il est possible d'inclure des commentaires multi-ligne avec les délimiteurs #{ et }#.

Exemple :

```
# Ceci est un commentaire sur une ligne
#{ Ceci est un commentaire
  sur plusieurs lignes. }#
```

1.2.3 Les valeurs

Les variables en M prennent soit leur valeur sur les flottants, soit ont la valeur *indefini*. Les valeurs de type booléen sont représentées par les flottants 0 et 1, respectivement pour *faux* et *vrai*.

Les résultats suivants peuvent être observés en exécutant le script disponible dans l'exemple sur le *Calcul des valeurs*.

Calcul booléen

Les opérations booléennes standard (et, ou, non) comportent sur les flottants de façon standard. Toute valeur flottante différente de 0 sera considérée comme vraie dans le cas d'un calcul booléen (10 ou 0 = 1). Les calculs booléens impliquant la valeur *indefini* ont un comportement spécifique :

- indefini ou indefini = indefini
- indefini ou b = (0 si b = 0, 1 sinon)
- b ou indefini = (0 si b = 0, 1 sinon)
- indefini et indefini = indefini
- b et indefini = indefini
- indefini et b = indefini
- non indefini = indefini

Les comparaisons (=, <, >, <=, >=) renvoient soit 0 soit 1 lorsque des valeurs définies sont comparées. Si l'une des valeurs comparée est *indefini*, alors le résultat est également *indefini*.

Note : même l'égalité *indefini* = *indefini* renvoie *indefini*. Pour vérifier si une valeur est définie ou non, il faut utiliser la fonction *present* qui renvoie 0 si la valeur est indéfinie et 1 sinon.

Calcul numérique

Les opérations arithmétiques standard (+, -, *, /) se comportent sur les flottants de façon standard, à l'exception de la division d'un flottant par zero qui renvoie toujours 0. Les calculs impliquant la valeur `indefini` ont un comportement spécifique :

```
— indefini + indefini = indefini
— indefini + n = n
— n + indefini = n
— indefini - indefini = indefini
— indefini - n = -n
— n - indefini = n
— indefini * indefini = indefini
— indefini * n = indefini
— n * indefini = indefini
— indefini / indefini = indefini
— indefini / n = indefini
— n / indefini = indefini
```

Pour résumer, seules les opérations d'addition et de soustraction avec une valeur renvoient autre chose que `indefini`.

1.3 Les fonctions

Voici la liste de toutes les fonctions standard du M. Leur utilisation est présentée dans l'exemple sur [Les fonctions](#).

1.3.1 `abs(X)`

Cette fonction prend un argument et renvoie sa valeur absolue. Si `X` est `indefini`, alors `abs(X) = indefini`.

1.3.2 `afficher(X) / afficher « texte »`

Cette fonction affiche sur la sortie standard la valeur de l'expression en argument.

1.3.3 `arr(X)`

Cette fonction prend un argument et renvoie l'entier le plus proche, ou `indefini` s'il est `indefini`.

1.3.4 `attribut(X, attribut)`

Cette fonction prend un argument `X` et un nom d'attribut `a` et retourne la valeur associée à l'attribut `a` de `X`.

1.3.5 `champ_evenement(X, champ)`

Cette fonction prend un argument `X` et un nom de champ d'événement `champ`. La valeur `X` correspond à l'identifiant de l'événement. %TODO : référence chap événement Si le `champ` correspond à une valeur, `champ_evenement` la retourne. Si le `champ` correspond à une variable, `champ_evenement` retourne sa valeur.

Note : `champ_evenement` peut être utilisée pour assigner des valeurs et des références à des événements. Exemple :

```
evenement
: valeur ev_val
: variable ev_var;

# Associe la valeur 42 au champ ev_val de l'événement 0.
champ_evenement (0,ev_val) = 42;
# Associe la variable X au champ ev_var de l'événement 0.
champ_evenement (0,ev_var) reference X;
```


1.3.6 inf(X)

Cette fonction prend un argument et renvoie l'entier inferieur le plus proche, ou `indefini` s'il est `indefini`.

1.3.7 max(X, Y)

Cette fonction prend deux arguments et renvoie la plus grande valeur des deux. Si les deux valeurs sont `indefinies`, alors la fonction renvoie `indefini`. Si X est défini et Y est `indefini`, alors $\text{max}(X, Y) = \text{max}(X, 0)$. De même, si X est `indefini` et Y est défini, alors $\text{max}(X, Y) = \text{max}(0, Y)$.

1.3.8 meme_variable(X, Y)

Cette fonction prend en argument deux variables et vérifie s'il s'agit de la même variable. Elle est notamment utilisée dans les itérateurs de variables pour effectuer des traitements spécifiques.

1.3.9 multimax(X, TAB)

Cette fonction prend deux arguments X une valeur et TAB un tableau de valeurs. Elle calcule la plus grande valeur contenue dans TAB entre 0 et X - 1. Si X est `indefini` ou $X \leq 0$, alors $\text{multimax}(X, \text{TAB}) = \text{indefini}$. X peut être plus grand que la taille de TAB, auquel cas `multimax` calculera le maximum de TAB.

1.3.10 min(X, Y)

Cette fonction prend deux arguments et renvoie la plus petite valeur des deux. Si les deux valeurs sont `indefinies`, alors la fonction renvoie `indefini`. Si X est défini et Y est `indefini`, alors $\text{min}(X, Y) = \text{min}(X, 0)$. De même, si X est `indefini` et Y est défini, alors $\text{min}(X, Y) = \text{min}(0, Y)$.

1.3.11 nb_evenements()

Cette fonction renvoie le nombre total d'événements.

1.3.12 null(X)

Cette fonction prend un argument X et renvoie 1 si X est égal à 0, 0 si X est défini et différent de 0, et `indefini` s'il est `indefini`.

Cette fonction est strictement équivalente à l'expression $(X = 0)$

1.3.13 positif(X)

Cette fonction prend un argument X et renvoie 1 si X est **strictement** supérieur à 0, 0 si X est inferieur ou égal à 0, et `indefini` si X est `indefini`.

Cette fonction est strictement équivalente à l'expression $X > 0$.

1.3.14 positif_ou_nul(X)

Cette fonction prend un argument X et renvoie 1 si X est supérieur ou égale à 0, 0 si X est strictement inferieur ou égal à 0, et `indefini` si X est `indefini`.

Cette fonction est strictement équivalente à l'expression $X \geq 0$.

1.3.15 present(X)

Cette fonction prend un argument X et renvoie 0 s'il est `indefini` et 1 sinon.

Note : cette fonction est la seule façon de tester si une valeur est égale à `indefini` car l'expression booléenne $(\text{indefini} = \text{indefini})$ est égale à `indefini`.

1.3.16 somme(X, Y, ...)

Cette fonction n'a pas de limite d'arguments. Elle calcule leur somme. Si aucun argument n'est donné à la fonction `somme`, elle renvoie `0`. Si tous ses arguments sont `indefinis`, alors elle renvoie la valeur `indefini`.

1.3.17 supzero(X)

Cette fonction prend un argument `X` et renvoie sa valeur s'il est strictement supérieur à `0`, sinon il renvoie `indefini`.

1.3.18 taille(TAB)

Cette fonction prend en argument soit une variable, soit un tableau. S'il s'agit d'une variable, `taille` renvoie `1`, sinon elle renvoie la taille du tableau. Elle échoue si l'argument est une constante ou une valeur.

1.3.19 type(V, type)

Cette fonction prend en argument une variable et un type. Elle renvoie `1` si `V` est du type donné, `0` sinon.

1.4 Les fichiers IRJ

1.4.1 Syntaxe

Les fichiers IRJ sont des fichiers de test faisant coorespondre entrées et sorties du calcul d'un programme M. Chaque fichier IRJ est divisé en 7 parties, plus 3 parties optionnelles, et est terminé par la ligne `##`.

- `NOM` : le nom du test.
- `ENTREES-PRIMITIF` : les valeurs des variables d'entrées au début du calcul. A chaque variable `VAR` est associée une valeur `val` entière ou décimale sur une ligne de la forme `VAR/val`. Les variables d'entrées qui ne sont pas définies dans le fichier IRJ seront initialisées à `indefini`.
- `CONTROLES-PRIMITIF` : à documenter
- `RESULTATS-PRIMITIF` : les valeurs des variables restituées à la fin du calcul. Comme pour les variables d'entrées, elles sont décrites sous la forme `VAR/val`. Les valeurs restituées absentes du fichiers IRJ sont supposées avoir la valeur `indefini` à la fin du calcul.
- `ENTREES-CORRECTIF` : à documenter
- `CONTROLES-CORRECTIF` : à documenter
- `RESULTATS-CORRECTIF` : à documenter
- `ENTREES-RAPPELS` : partie optionnelle, à documenter
- `CONTROLES-RAPPELS` : partie optionnelle, à documenter
- `RESULTATS-RAPPELS` : partie optionnelle, à documenter

Voici un exemple de fichier IRJ minimal

```
#NOM
MON-TEST
#ENTREES-PRIMITIF
X/0.0
#CONTROLES-PRIMITIF
#RESULTATS-PRIMITIF
Y/1
Z/2.0
#ENTREES-CORRECTIF
#CONTROLES-CORRECTIF
#RESULTATS-CORRECTIF
##
```

De nombreux exemples de fichiers IRJ sont disponibles dans le dossier `tests/`.

1.4.2 Utilisation

Trois utilitaires sont dédiés à l'utilisation des fichiers IRJ.

IRJ checker

Les sources de mlang mettent à disposition un code de vérification de fichiers IRJ. Après compilation, il peut être exécuté via la commande :

```
$ dune exec -- irj_checker
```

Interpreteur

Le binaire mlang intègre un interpreteur de M qui prend en entrée un ou plusieurs fichiers M ainsi qu'un fichier IRJ. Après compilation, l'interpreteur peut être exécuté via la commande :

```
$ dune exec -- mlang test.m -A application -b interpreter --mpp_function cible_dentree -  
↪ -dgfip_options='' -r test.irj
```

L'option -r peut être remplacée par -R pour tester l'ensemble des tests d'un dossier complet.

Fichiers C

Il est également possible d'utiliser les fichiers IRJ comme entrée du code C généré à partir d'une compilation de code M par mlang. Ce traitement est effectué dans le cas du backend dgfip_c de mlang, dont le code est disponible dans `examples/dgfip_c/ml_primitif`. La calcullette d'une année donnée peut être compilée via :

```
$ make YEAR=year compile_dgfip_c_backend
```

Les scripts présents dans `examples/dgfip_c/ml_primitif/ml_driver` permettent d'interfacer les fichiers IRJ avec les valeurs C de type `T_irdata` contenant le TGV (Tableau Général des Variables). La commande suivante permet de tester la calcullette d'une année sur l'ensemble des tests fuzzés de la dite année.

```
$ make test_dgfip_c_backend
```

L'utilisation du script `cal` utilisé dans cette règle est documentée dans `examples/dgfip_c/README.md`.

1.5 Le compilateur MLang

1.5.1 Installer

MLang est implanté en OCaml. L'utilisation du gestionnaire de paquets OCaml `opam` est fortement recommandée. Vous pouvez l'installer via votre gestionnaire de paquet préféré s'il distribue `opam`, ou bien en vous référant à la [documentation d'opam](#). Mlang a également quelques autres dépendances, dont une vers la librairie de calcul de flottants MPFR. Si vous êtes sous Debian, vous pouvez simplement utiliser la commande suivante pour installer toutes les dépendances externes à OCaml :

```
$ sudo apt install \  
    libgmp-dev \  
    libmpfr-dev \  
    git \  
    patch \  
    unzip \  
    bubblewrap \  
    
```

(suite sur la page suivante)

```
bzip2 \  
opam
```

Si vous n'avez jamais utilisé `opam`, commencez par lancer :

```
$ opam init  
$ opam update
```

Enfin, vous pouvez initialiser le projet `mlang` avec

```
$ make init
```

Note pour les utilisateurs d'opam confirmés : la commande `make init` crée un switch local où seront installées les dépendances OCaml de `mlang`.

Cette commande initialise le dossier `ir-calcul` dans lequel sont poussés le code de calcul primitif de l'impôt sur le revenu.

Si besoin, la commande

```
$ make deps
```

réinstallera les dépendances OCaml et mettra à jour le dossier `ir-calcul`.

Une fois compilé, vous pouvez soit appeler `mlang` via la commande :

```
$ opam exec -- mlang
```

tant que vous êtes dans le dossier depuis lequel vous avez compilé, soit l'installer localement avec la commande :

```
opam install ./mlang.opam
```

1.5.2 Utiliser MLang

Le binaire `mlang` prend en argument le fichier *M* à exécuter. Il peut également prendre en argument une liste de fichiers, auquel cas leur traitement sera équivalent au traitement d'un seul et même fichier dans lequel serait concaténé le contenu de chaque fichier.

Les options principales sont :

- `-A` : le nom de l'application à traiter ;
- `-b` : le mode d'utilisation, ou `backend` ;
- `--mpp_function` : le nom de la fonction principale à traiter.

Mode interpréteur

Le mode interpréteur de `mlang` utilise un fichier *IRJ* (voir [Les fichiers IRJ](#)) pour exécuter le code *M* directement depuis sa représentation abstraite.

Voici une commande simple pour invoquer l'interpréteur :

```
$ mlang test.m \  
  -A mon_application \  
  -b interpreter \  
  --mpp_function hello_world \  
  --run_test test.irj \  
  --without_dfgip_m
```

Mode transpilation

Le mode transpilation de mlang permet de traduire le code *M* dans un autre langage. En 2025, seul le langage C est supporté. Voici une commande simple pour traduire un fichier *M* en C :

```
$ mlang test.m \
  -A mon_application \
  -b dgfip_c \
  --mpp_function hello_world
  --dgfip_options=''
  --output output/mon-test.c
```

NB : le dossier output doit avoir été créé en amont.

Options DGFIP

Les options DGFIP sont à usage interne. Elles sont spécifiées dans l'option `--dgfip_options`.

```
-b VAL
  Set application to "batch" (b0 = normal, b1 = with EBCDIC sort)

-D Generate labels for output variables

-g Generate for test (debug)

-I Generate immediate controls

-k VAL (absent=0)
  Number of debug files

-L Generate calls to ticket function

-m VAL (absent=1991)
  Income year

-O Optimize generated code (inline min_max function)

-o Generate overlays

-P Primitive calculation only

-r Pass TGV pointer as register in rules

-R Set application to both "iliad" and "pro"

-s Strip comments from generated output

-S Generate separate controls

-t Generate trace code

-U Set application to "cfir"

-x Generate cross references
```

(suite sur la page suivante)

- X Generate **global** extraction
- Z Colored output **in** chainings

1.5.3 Comportement de Mlang

Le compilateur Mlang effectue son traitement en quatre étapes :

- la traduction dans un format abstrait interne ;
- un pré-traitement pour le simplifier ;
- une vérification pour analyser la cohérence du code ;
- le traitement du code M, que ce soit son interprétation ou sa compilation.

Le module Driver (et plus précisément la fonction `Driver.main`) correspond au point d'entrée de mlang.

Traduction

Le langage M est parsé selon les règles spécifiées dans *La syntaxe du M*. Elle est effectuée par les modules `Mparser`, `Mlexer` et `Parse_utils`.

Pré-traitement

Le prétraitement est une opération purement syntaxique. Elle est effectuée par les modules `Expander` et `Mir`. Son but est triple :

- éliminer les constructions relatives aux applications non-sélectionnées ;
- remplacer les constantes par leur valeur numérique ;
- remplacer les expressions numériques débutant par `somme` avec des additions ;
- éliminer les `<multi-formule>`s en les remplaçant par des séries de `<formule>`s.

Toute substitution transformant un programme M syntaxiquement valide en un texte ne correspondant à aucun programme M provoque l'échec du traitement.

Cas des applications

On élimine du programme M :

- les déclarations des applications non-sélectionnées ;
- les déclarations des enchaîneurs ne spécifiant pas une application sélectionnée ;
- les déclarations des règles ne spécifiant pas une application sélectionnée ;
- les déclarations des vérifications ne spécifiant pas une application sélectionnée ;
- les déclarations des cibles ne spécifiant pas une application sélectionnée.

Dans les déclarations des règles spécifiant une application sélectionnée, on élimine les enchaîneurs qui ne sont plus déclarés dans le programme M obtenu.

Cas des constantes

Pour prétraiter un programme, on le parcourt du début à la fin. Pour chaque `<variable>` rencontrée, si elle correspond à une constante défini précédemment, alors il est remplacé dans le programme par la valeur numérique correspondante.

Un intervalle de la forme `<naturel:début>...<symbole:const>` est converti par substitution de la constante `const` en l'intervalle `<naturel:début>...<naturel:fin>`, avec `fin` le naturel correspondant à `const`.

Exemple : considérons le programme suivant :

```
fin : const = 10;
...
```

(suite sur la page suivante)

(suite de la page précédente)

```
pour i = 9-fin :
  Bi = Bi + fin;
```

Par substitution de la constante *fin*, il est remplacé dans un premier temps remplacée par le programme :

```
fin : const = 10;`
...
pour i = 9..10 :
  Bi = Bi + fin;
```

puis dans un second temps par le programme :

```
fin : const = 10;
...
pour i = 9..10 :
  Bi = Bi + 10;
```

Interprétation des indices

Pour rappel, les *indices* ont la forme suivante :

```
<indices> ::= <indice> ; ...
<indice> ::= <minuscule> = <intervalle> , ...
```

avec :

- <minuscule> ::= [a-z]
- <majuscule> ::= [A-Z]
- <intervalle> ::= <majuscule>+ | <majuscule> .. <majuscule> | <naturel> (.. <naturel> | - <variable>)?

Chaque <indice> associe à une lettre minuscule une série de chaînes de caractères de même taille. Cette série est composée de la succession de chaque à droite du symbole =.

Les séries associées aux sont définis comme suit :

- <majuscule>+ est la série composée de chacune des majuscules prises séparément, donc de taille 1 (*par exemple* : AXF représente la série A, X, F);
- <majuscule:/début/>****<majuscule:/fin/> est la série composée de toutes les majuscules comprises entre /début/ et /fin/, bornes comprises, suivant l'ordre alphabétique, donc de taille 1 (*par exemple* : A..D représente la série A, B, C, D);
- <naturel:début>..<naturel:fin> est la série composée de tous les nombres naturels entre début et fin, bornes comprises, la taille des éléments étant égale à la taille du plus grand naturel en base 10; les naturels trop petits pour avoir la taille requise sont complétés par des 0 à gauche (*par exemple* : 9..11 représente la série 09, 10, 11);
- <naturel>-<variable> est converti lors du prétraitement des constantes en un intervalle de la forme <naturel>..<naturel>.

Cas des expressions somme(...)

Pour une expression numérique `somme ( <indice:ind>: <expression numerique:expr> )`, l'expression *expr* sera remplacée par la somme des expressions construites ainsi : pour chaque chaîne de caractères de la série introduite par *ind*, les symboles apparaissant dans *expr* sont remplacés par ceux dans lesquels la lettre minuscule de /ind/ est substitué par la chaîne de caractère.

La somme ainsi générée est parenthésée.

Une expression numérique `somme ( <indice:ind> ; <indices:inds> : <expression numérique:expr> )`, est remplacée par la somme des expressions `somme <indices:inds> : expr'> )` avec `expr'` prenant sa valeur dans la série `sub(ind, expr)`.

La procédure est ensuite appliquée récursivement à cette somme.

La somme ainsi générée est parenthésée.

Notons que les sous-sommes récursivement produites n'ont pas besoin de l'être car l'addition est associative.

On applique cette transformation à toutes les expressions `somme(...)` tant qu'il en existe dans le programme.

Exemple. Considérons l'expression suivante :

— `somme(i = XZ ; j = 9..10 : Bi + Bj)`

Elle est dans un premier temps remplacée par la somme suivante :

— `(somme(j = 9..10 : BX + Bj) + somme(j = 9..10 : BZ + Bj))`

puis dans un second temps par la somme :

— `(BX + B09 + BX + B10 + BZ + B09 + BZ + B10)`

On remarque que seule l'expression globale est parenthésée car l'addition est associative.

Cas des multi-formules

On commence par substituer toutes les expressions `somme(...)` apparaissant dans les multi-formules.

Puis on transforme les multi-formules en séries de formules en suivant la méthode décrite ci-dessous.

Pour chaque multi-formule rencontrée, de la forme `pour <indices> : <formule>`, dès que les constantes apparaissant dans les ont été substitués, on remplace cette multi-formule par la série de formules générée de la manière suivante.

Pour une multi-formule `pour <indice:ind> : <formule:f>`, la formule `f` sera remplacée par la série des formules construites ainsi : pour chaque chaîne de caractères de la série introduite par `ind`, les symboles apparaissant dans `f` sont remplacés par ceux dans lesquels la lettre minuscule de `ind` est substitué par la chaîne de caractère.

On notera `sub(ind, f)` la série constituée des formules ainsi régérées.

Une multi-formule `pour <indice:ind> ; <indices:inds> : <formule:f>`, est remplacée par la série des multi-formules `pour <indices:inds> : f'>` avec `f'` prenant sa valeur dans la série `sub(ind, f)`.

La procédure est ensuite appliquée récursivement à ces multi-formules.

Exemple. Considérons la multi-formule suivante :

```
pour i = XZ ; j = 9..10 :
  Bi = Bi + Bj;
```

Elle est dans un premier temps remplacée par la série suivante :

```
pour j = 9..10 :
  BX = BX + Bj;

pour j = 9..10 :
  BZ = BZ + Bj;
```

Puis dans un second temps par la série des formules :

```
BX = BX + B09;
BX = BX + B10;
BZ = BZ + B09;
BZ = BZ + B10;
```


Vérification de cohérence

De nombreuses constructions sont valides à la traduction, mais brisent certains invariants nécessaires à la bonne exécution du code : double déclaration d'attributs, nom de variable déjà utilisé, variable mal typée... L'ensemble de ces vérifications est accessible dans le module `Validator` du frontend. Le sous module `Validator.Err` définit l'ensemble des erreurs levées par cette étape de vérification.

NB : seule la première erreur rencontrée par le validateur est levée.

Traitement

Interpreteur

L'interpreteur utilise la représentation interne du code M pour lancer le calcul à partir d'un fichier IRJ (voir [Les fichiers IRJ](#)). L'interprétation est effectuée par le module `Test_interpreter`.

Transpilation

La transpilation traduit le code dans le langage spécifié (en 2025, seul le C est transpilable). La transpilation est effectuée dans le module `Bir_to_dgfip_c`.

2.1 Calcul des valeurs

2.1.1 Fichier test : test.m

```
espace_variables GLOBAL : par_defaut;
domaine regle mon_domaine_de_regle: par_defaut;
domaine verif mon_domaine_de_verifications: par_defaut;
application mon_application;
variable saisie : attribut mon_attribut;
variable calculee : attribut mon_attribut;
X : saisie mon_attribut = 0 alias AX : "";

cible indefini_test:
application : mon_application;
afficher "Bonjour, monde, X = ";
afficher (X);
afficher " !\n";
# Addition
afficher "X + 1 = ";
afficher (X + 1);
afficher " !\n";
afficher "1 + X = ";
afficher (1 + X);
afficher " !\n";
afficher "X + X = ";
afficher (X + X);
afficher " !\n";
# Soustraction
afficher "X - 1 = ";
afficher (X - 1);
afficher " !\n";
```

(suite sur la page suivante)

```
afficher "1 - X = ";
afficher (1 - X);
afficher " !\n";
afficher "X - X = ";
afficher (X - X);
afficher " !\n";
# Multiplication
afficher "X * 1 = ";
afficher (X * 1);
afficher " !\n";
afficher "1 * X = ";
afficher (1 * X);
afficher " !\n";
afficher "X * X = ";
afficher (X * X);
afficher " !\n";
# Division
afficher "X / 1 = ";
afficher (X / 1);
afficher " !\n";
afficher "1 / X = ";
afficher (1 / X);
afficher " !\n";
afficher "1 / 0 = ";
afficher (1 / 0);
afficher " !\n";
afficher "X / 0 = ";
afficher (X / 0);
afficher " !\n";
afficher "X / X = ";
afficher (X / X);
afficher " !\n";
# Comparaisons
afficher "(X = 0) = ";
afficher (X = 0);
afficher " !\n";
afficher "(X = X) = ";
afficher (X = X);
afficher " !\n";
afficher "(X >= 0) = ";
afficher (X >= 0);
afficher " !\n";
afficher "(X >= X) = ";
afficher (X >= X);
afficher " !\n";
afficher "(X > 0) = ";
afficher (X > 0);
afficher " !\n";
afficher "(X > X) = ";
afficher (X > X);
afficher " !\n";
afficher "(X <= 0) = ";
```

(suite sur la page suivante)

(suite de la page précédente)

```

afficher (X <= 0);
afficher " !\n";
afficher "(X < 0) = ";
afficher (X < 0);
afficher " !\n";
afficher "(X <= X) = ";
afficher (X <= X);
afficher " !\n";
afficher "(X < X) = ";
afficher (X < X);
afficher " !\n";
# Opérations booléennes
afficher "(X et X) = ";
afficher (X et X);
afficher " !\n";
afficher "(X et 1) = ";
afficher (X et 1);
afficher " !\n";
afficher "(X ou X) = ";
afficher (X ou X);
afficher " !\n";
afficher "(X ou 1) = ";
afficher (X ou 1);
afficher " !\n";
afficher "non X = ";
afficher (non X);
afficher " !\n";
# Vérifier la définition d'une valeur
afficher "present(X) = ";
afficher (present(X));
afficher " !\n";

```

2.1.2 Fichier irj : test.irj

```

#NOM
MON-TEST
#ENTREES-PRIMITIF
#CONTROLES-PRIMITIF
#RESULTATS-PRIMITIF
#ENTREES-CORRECTIF
#CONTROLES-CORRECTIF
#RESULTATS-CORRECTIF
##

```

2.1.3 Commande

```

mlang test.m \
    --without_dfgip_m \
    -A mon_application \
    --mpp_function indefini_test \
    --run_test test.irj

```

2.2 Les fonctions

2.2.1 Fichier test : test.m

```

espace_variables GLOBAL : par_defaut;
domaine regle mon_domaine_de_regle: par_defaut;
domaine verif mon_domaine_de_verifications: par_defaut;
application mon_application;
variable calculee : attribut mon_attribut;

evenement
: valeur ev_val
: variable ev_var;

X : calculee mon_attribut = 0 : "" type REEL;
Y : calculee mon_attribut = 1 : "";
TAB : tableau[10] calculee mon_attribut = 2 : "" type ENTIER;

cible init_tab:
application : mon_application;
iterer : variable I : entre 0..(taille(TAB) - 1) increment 1 : dans (
  TAB[I] = I;
)

cible reinit_tab:
application : mon_application;
iterer : variable I : entre 0..(taille(TAB) - 1) increment 1 : dans (
  TAB[I] = indefini;
)

cible test_abs:
application : mon_application;
afficher "\n__ABS__";
afficher indenter(2);
afficher "\nabs(indefini) = ";
afficher (abs(indefini));
afficher "\nabs(1) = ";
afficher (abs(1));
afficher "\nabs(-1) = ";
afficher (abs(-1));
afficher "\n";
afficher indenter(-2);

cible test_arr:
application : mon_application;
afficher "\n__ARR__";
afficher indenter(2);
afficher "\narr(indefini) = ";
afficher (arr(indefini));
afficher "\narr(1.8) = ";
afficher (arr(1.8));
afficher "\ arr(-1.7) = ";
afficher (arr(-1.7));

```

(suite sur la page suivante)

(suite de la page précédente)

```

afficher "\n";
afficher indenter(-2);

cible test_attribut:
application : mon_application;
afficher "\n__ATTRIBUT__";
afficher indenter(2);
afficher "\nattribut(X, mon_attribut) = ";
afficher (attribut(X, mon_attribut));
afficher "\nattribut(TAB, mon_attribut) = ";
afficher (attribut(TAB, mon_attribut));
afficher "\n";
afficher indenter(-2);

cible test_champ_evenement_base:
application : mon_application;
afficher indenter(2);
afficher "\nchamp_evenement(0, ev_val) = ";
afficher (champ_evenement (0,ev_val));
afficher "\nchamp_evenement(0, ev_var) = ";
afficher (champ_evenement (0,ev_var));
afficher "\n";
afficher indenter(-2);

cible test_champ_evenement:
application : mon_application;
afficher "\n__CHAMP_EVENEMENT__";
afficher indenter(2);
arranger_evenements
: ajouter 1
: dans (
    afficher "\nAvant d'initialiser les champs de l'événement:";
    calculer cible test_champ_evenement_base;
    champ_evenement (0,ev_var) reference X;
    champ_evenement (0,ev_val) = 42;
    X = 2;
    afficher "\nAprès avoir initialisé les champs de l'événement:";
    calculer cible test_champ_evenement_base;
    X = indefini;
)
afficher indenter(-2);

cible test_inf:
application : mon_application;
afficher "\n__INF__";
afficher indenter(2);
afficher "\ninf(indefini) = ";
afficher (inf(indefini));
afficher "\ninf(1.8) = ";
afficher (inf(1.8));
afficher "\ninf(-1.7) = ";
afficher (inf(-1.7));

```

(suite sur la page suivante)

```
afficher "\n";
afficher indenter(-2);

cible test_max:
application : mon_application;
afficher "\n__MAX__";
afficher indenter(2);
afficher "\nmax(indefini, indefini) = ";
afficher (max(indefini, indefini));
afficher "\nmax(-1, indefini) = ";
afficher (max(-1, indefini));
afficher "\nmax(indefini, -1) = ";
afficher (max(indefini, -1));
afficher "\nmax(1, indefini) = ";
afficher (max(1, indefini));
afficher "\n";
afficher indenter(-2);

cible test_meme_variable:
application : mon_application;
afficher "\n__MEME_VARIABLE__";
afficher indenter(2);
afficher "\nmeme_variable(X,X) = ";
afficher (meme_variable(X,X));
afficher "\nmeme_variable(X,TAB) = ";
afficher (meme_variable(X,TAB));
afficher "\nmeme_variable(TAB[0],TAB) = ";
afficher (meme_variable(TAB[0],TAB));
afficher "\n";
afficher indenter(-2);

cible test_min:
application : mon_application;
afficher "\n__MIN__";
afficher indenter(2);
afficher "\nmin(indefini, indefini) = ";
afficher (min(indefini, indefini));
afficher "\nmin(1, indefini) = ";
afficher (min(1, indefini));
afficher "\nmin(indefini, 1) = ";
afficher (min(indefini, 1));
afficher "\nmin(-1, indefini) = ";
afficher (min(-1, indefini));
afficher "\n";
afficher indenter(-2);

cible test_multimax_base:
application : mon_application;
afficher indenter(2);
afficher "\nmultimax(indefini, TAB) = ";
afficher (multimax(indefini, TAB));
afficher "\nmultimax(7, TAB) = ";
```

(suite sur la page suivante)

(suite de la page précédente)

```

afficher (multimax(7, TAB));
afficher "\nmultimax(taille(TAB) + 1, TAB) = ";
afficher (multimax(taille(TAB) + 1, TAB));
afficher "\nmultimax(0, TAB) = ";
afficher (multimax(0, TAB));
afficher "\nmultimax(-1, TAB) = ";
afficher (multimax(-1, TAB));
afficher indenter(-2);

cible test_multimax:
application : mon_application;
afficher "\n__MULTIMAX__";
afficher indenter(2);
afficher "\nAvant initialisation du tableau :";
calculer cible test_multimax_base;
calculer cible init_tab;
afficher "\nAprès initialisation du tableau :";
calculer cible test_multimax_base;
calculer cible reinit_tab;
afficher "\n";
afficher indenter(-2);

cible test_nb_evenements_base:
application : mon_application;
afficher indenter(2);
afficher "\nnb_evenements() = ";
afficher (nb_evenements ());
afficher "\n";
afficher indenter(-2);

cible test_nb_evenements:
application : mon_application;
afficher "\n__NB_EVENEMENTS__";
afficher indenter(2);
afficher "\nAvant la définition d'un événement :";
calculer cible test_nb_evenements_base;
arranger_evenements
: ajouter 1
: dans (
    afficher "\nPendant la définition d'un événement :";
    calculer cible test_nb_evenements_base;
)
afficher "\nAprès la définition d'un événement :";
calculer cible test_nb_evenements_base;
afficher indenter(-2);

cible test_null:
application : mon_application;
afficher "\n__NULL__";
afficher indenter(2);
afficher "\nnull(indefini) = ";
afficher (null(indefini));

```

(suite sur la page suivante)

```
afficher "\nnull(0) = ";
afficher (null(0));
afficher "\nnull(1) = ";
afficher (null(1));
afficher "\n";
afficher indenter(-2);

cible test_positif:
application : mon_application;
afficher "\n__POSITIF__";
afficher indenter(2);
afficher "\npositif(indefini) = ";
afficher (positif(indefini));
afficher "\npositif(0) = ";
afficher (positif(0));
afficher "\npositif(1) = ";
afficher (positif(1));
afficher "\npositif(-1) = ";
afficher (positif(-1));
afficher "\n";
afficher indenter(-2);

cible test_positif_ou_nul:
application : mon_application;
afficher "\n__POSITIF OU NUL__";
afficher indenter(2);
afficher "\npositif_ou_nul(indefini) = ";
afficher (positif_ou_nul(indefini));
afficher "\npositif_ou_nul(0) = ";
afficher (positif_ou_nul(0));
afficher "\npositif_ou_nul(1) = ";
afficher (positif_ou_nul(1));
afficher "\npositif_ou_nul(-1) = ";
afficher (positif_ou_nul(-1));
afficher "\n";
afficher indenter(-2);

cible test_present:
application : mon_application;
afficher "\n__PRESENT__";
afficher indenter(2);
afficher "\npresent(indefini) = ";
afficher (present(indefini));
afficher "\npresent(0) = ";
afficher (present(0));
afficher "\npresent(1) = ";
afficher (present(1));
afficher "\n";
afficher indenter(-2);

cible test_somme:
application : mon_application;
```

(suite sur la page suivante)

(suite de la page précédente)

```
afficher "\n__SOMME__";
afficher indenter(2);
afficher "\nsomme() = ";
afficher (somme());
afficher "\nsomme(indefini) = ";
afficher (somme(indefini));
afficher "\nsomme(1, indefini) = ";
afficher (somme(1, indefini));
afficher "\nsomme(1, 2, 3, 4, 5) = ";
afficher (somme(1, 2, 3, 4, 5));
afficher "\n";
afficher indenter(-2);
```

```
cible test_supzero:
application : mon_application;
afficher "\n__SUPZERO__";
afficher indenter(2);
afficher "\nsupzero(indefini) = ";
afficher (supzero(indefini));
afficher "\nsupzero(42) = ";
afficher (supzero(42));
afficher "\nsupzero(-1) = ";
afficher (supzero(-1));
afficher "\nsupzero(0) = ";
afficher (supzero(0));
afficher "\n";
afficher indenter(-2);
```

```
cible test_taille:
application : mon_application;
afficher "\n__TAILLE__";
afficher indenter(2);
afficher "\ntaille(TAB) = ";
afficher (taille(TAB));
afficher "\ntaille(X) = ";
afficher (taille(X));
afficher "\n";
afficher indenter(-2);
```

```
cible test_type:
application : mon_application;
afficher "\n__TYPE__";
afficher indenter(2);
afficher "\ntype(X, REEL) = ";
afficher (type(X, REEL));
afficher "\ntype(X, ENTIER) = ";
afficher (type(X, ENTIER));
afficher "\ntype(TAB, ENTIER) = ";
afficher (type(TAB, ENTIER));
afficher "\ntype(Y, ENTIER) = ";
afficher (type(Y, ENTIER));
afficher "\ntype(Y, REEL) = ";
```

(suite sur la page suivante)

```
afficher (type(Y, REEL));
afficher "\ntype(Y, BOOLEEN) = ";
afficher (type(Y, BOOLEEN));
afficher "\ntype(Y, DATE_AAAA) = ";
afficher (type(Y, DATE_AAAA));
afficher "\ntype(Y, DATE_JJMMAAAA) = ";
afficher (type(Y, DATE_JJMMAAAA));
afficher "\ntype(Y, DATE_MM) = ";
afficher (type(Y, DATE_MM));
afficher "\n";
afficher indenter(-2);

cible fun_test:
application : mon_application;
# Abs
calculer cible test_abs;
# Arr
calculer cible test_arr;
# Attribut
calculer cible test_attribut;
# Champ evenement
calculer cible test_champ_evenement;
# Inf
calculer cible test_inf;
# Max
calculer cible test_max;
# Meme variable
calculer cible test_meme_variable;
# Min
calculer cible test_min;
# Multimax
calculer cible test_multimax;
# Null
calculer cible test_nb_evenements;
# Null
calculer cible test_null;
# Positif
calculer cible test_positif;
# Positif ou nul
calculer cible test_positif_ou_nul;
# Présent
calculer cible test_present;
# Somme
calculer cible test_somme;
# Supzero
calculer cible test_supzero;
# Taille
calculer cible test_taille;
# Type
calculer cible test_type;
```

2.2.2 Fichier irj : test.irj

```
#NOM
MON-TEST
#ENTREES-PRIMITIF
#CONTROLES-PRIMITIF
#RESULTATS-PRIMITIF
#ENTREES-CORRECTIF
#CONTROLES-CORRECTIF
#RESULTATS-CORRECTIF
##
```

2.2.3 Commande

```
mlang test.m \
    --without_dfgip_m \
    -A mon_application \
    --mpp_function fun_test \
    --run_test test.irj
```


CHAPITRE 3

Documentation développeur

— Index